

COMPLEX ENGINEERING PROBLEM

TITLE

Open-Source Repository Documentation Improvement

Rajvirsinh Dodiya [92400120478]

Harsh Kanjariya [92400120494]

Arun Busa [92400120614]

DEPARTMENT

COMPUTER ENGINEERING

ACADEMIC YEAR

2026-2027

SEMESTER

5TH

SUBJECT

OPEN SOURCE TECHNOLOGIES(01CE0526)

CLASS

EV4

BATCH

A

COMPLEX ENGINEERING PROBLEM - REPORT

RUBRICS FOR COMPLEX ENGINEERING PROBLEM

Sr. No.	Assessment Criteria	Excellent	Good	Satisfactory	Poor	Marks
1	Problem Analysis & Open Source Technology Selection	3 Marks: Clearly identifies the engineering problem, analyzes requirements and constraints, and selects appropriate Open Source technologies with proper justification.	2 Marks: Good problem analysis with suitable technology selection and minor gaps.	1 Mark: Basic analysis with limited justification of technology selection.	0 Marks: Inadequate problem analysis and inappropriate technology selection.	3
2	System Design, Implementation Strategy & Technical Analysis	4 Marks: Well-defined system architecture, appropriate technology stack, database, deployment strategy, version control, and comprehensive technical analysis including scalability, reliability, security.	3 Marks: Good system design and technical discussion with minor omissions.	1-2 Marks: Basic architecture and limited technical analysis.	0 Marks: Poor or missing system design and technical analysis.	4
3	Testing, Documentation & References	3 Marks: Comprehensive testing strategy, well-organized report, IEEE references, appropriate diagrams, and clear conclusions.	2 Marks: Good documentation with minor formatting or testing gaps.	1 Mark: Basic report with limited testing or references.	0 Marks: Incomplete report with poor documentation and missing references.	3
	Total					10

Sr. No.	Complex Engineering Problem Activity / Problem Title	COs	Date	R1(4)	R2(3)	R3(4)	Marks (10)	Signature
1	Open-Source Repository Documentation Improvement							

Notes / Instructions

1. CEP activity must be completed in a group of maximum 3 students.
2. Students must strictly follow the given deadlines for submission.
3. The report must be prepared in the prescribed format and submitted as a DOCX file on Google Classroom.
4. A hard copy of the report must be submitted as discussed.
5. The report should include problem analysis, design/architecture, implementation, results, and challenges faced.
6. All group members must contribute equally and be prepared for evaluation/presentation.
7. Plagiarism or copying from other groups will not be accepted.
8. Each group member must understand complete solution and should be able to explain their contribution during evaluation.

Ms. Priyanka Mangi
Subject Faculty

COMPLEX ENGINEERING PROBLEM - REPORT

Open-Source Repository Documentation Improvement

Course Name and Course Name: OPEN SOURCE TECHNOLOGIES(01CE0526)

Co's Mapped - CO1

SDG's Mapped - SDG 4, SDG 12, SDG 17

1. Aim

To analyse, design, and implement an authentic, comprehensive, and contributor-friendly documentation system for the selected open-source Todo application repository (<https://github.com/Aklilu-Mandefro/todo-app-in-javascript-html-and-css>) using Git, GitHub community standards, and structured Markdown architecture while strictly preserving the existing application source code.

2. Problem Statement

An existing open-source Todo web application repository built with HTML5, CSS3, vanilla JavaScript, Bootstrap 5, and Font Awesome lacked essential onboarding, technical, diagnostic, and contribution documentation. The original repository contained only a rudimentary 40-line README file with copied placeholder instructions, no architectural explanation of its client-side state handling or HTML5 localStorage mechanics, no local execution or web-server setup instructions, no diagnostic guidance for realistic runtime and caching failures, and no GitHub issue or pull request templates. The engineering problem requires systematically auditing the repository, architecting multi-tiered documentation, defining robust Git workflow conventions, and validating all links and local execution environments without altering the application's core implementation files.

3. Problem Analysis

a) What is the engineering problem?

The engineering problem is to eliminate developer and end-user onboarding barriers in an open-source client-side Todo web application. This involves identifying documentation deficits through repository inspection, designing a clean 3-tier documentation structure (Root, Operational Docs, Community Governance), documenting the data lifecycle between browser DOM events and localStorage, providing multi-platform local deployment instructions, establishing Conventional Commit and branch naming standards, and deploying issue/PR templates to facilitate collaborative software maintenance.

b) Why is this problem complex?

- Requires multiple engineering concepts
The task combines open-source repository management, Git-based version control, technical documentation, Markdown, issue management, contribution workflow, and collaborative software development.
- Requires analysis and decision making
The existing repository had to be inspected before making changes. Decisions were required regarding README organization, installation instructions, contribution workflow, issue-template structure, branch naming, commit conventions, and the final validation process.
- Involves design constraints
The documentation improvements had to preserve the existing project structure and application source code. The documentation also had to remain simple, readable,

COMPLEX ENGINEERING PROBLEM - REPORT

maintainable, compatible with Markdown rendering, and consistent with the actual repository files.

- Requires use of engineering tools
Git, GitHub, a text/Markdown editor, Command Prompt, and a web browser were used to perform and validate the repository documentation improvement workflow.
- Requires performance/security/safety consideration
Reliability was considered by ensuring that installation instructions and documented file paths corresponded to the project structure. Security considerations included avoiding passwords, API keys, access tokens, and other sensitive information in repository documentation. The contribution workflow also separates development changes from the main branch.

4. Knowledge and Tools Required

a) Engineering concepts used.

- 1) Git-Based Version Control
Git was used to create a dedicated documentation branch, track changes, commit documentation updates, push the branch, and maintain a traceable history of the implementation.
- 2) Open-Source Repository Management
GitHub fork and pull-request workflow was used to perform improvements in the student's own copy of the original open-source repository while preserving the original repository.
- 3) Technical Documentation and Collaborative Development
Markdown documentation, contribution guidelines, issue templates, branch conventions, and commit conventions were used to make the repository easier to understand, maintain, and contribute to.

b) Tools/Software/Hardware used

- Git
- GitHub
- Markdown/Text Editor
- Command Prompt
- Web Browser

5. Proposed Solution Design

a) Block Diagram / Architecture Diagram

COMPLEX ENGINEERING PROBLEM - REPORT

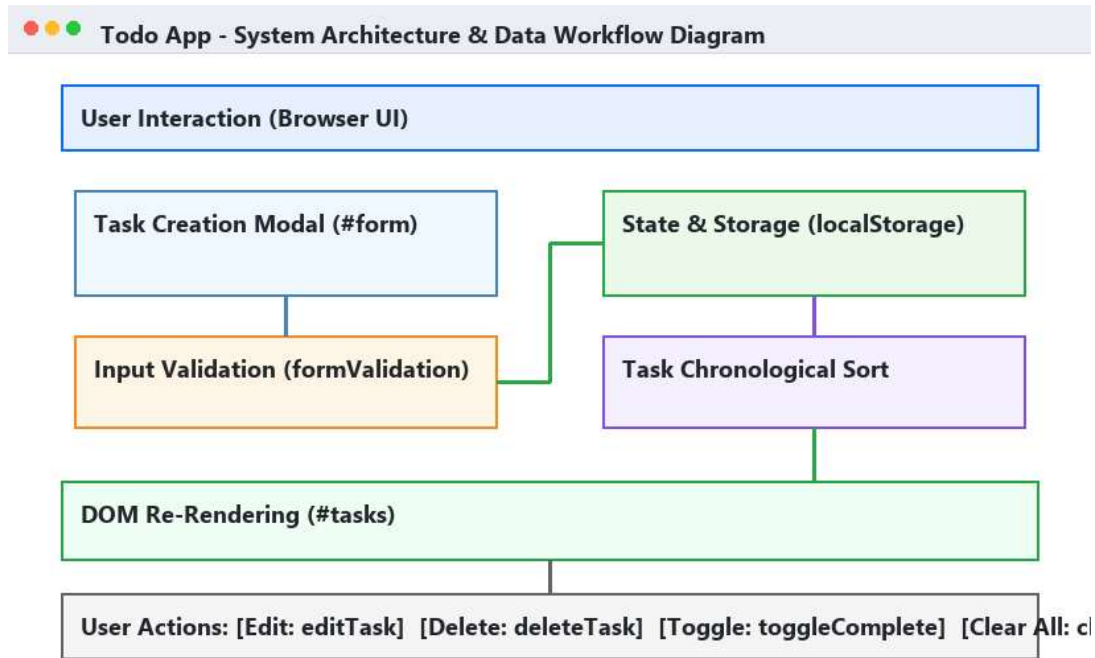


Fig. 13. System architecture and end-to-end data lifecycle diagram for the Todo application.

b) Brief Working

The documentation solution is structured into three cohesive tiers:

1. Top Tier (Root Documentation): An extensively rewritten README.md (316 lines) provides an architectural overview, verified feature matrix, technology stack breakdown, multiple launch methods (direct file execution and local HTTP servers), and step-by-step user instructions. It is paired with CONTRIBUTING.md (256 lines), which establishes a 10-stage contributor workflow, branch naming conventions, Conventional Commit standards, and browser-based testing guidelines.
2. Operational Tier (docs/ Directory): USER_GUIDE.md provides end users with a complete walkthrough of task creation modal inputs, mandatory validation constraints, chronological date sorting mechanics, in-place editing behavior, and localStorage retention boundaries. TROUBLESHOOTING.md presents a Problem / Possible Cause / Solution matrix addressing CDN font/script blockages, modal dismissal data loss during edit, private browsing storage loss, and browser cache busting.
3. Community Governance Tier (.github/ Directory): Standardized YAML front-matter issue templates for bug reports and feature requests ensure consistent defect reporting, while PULL_REQUEST_TEMPLATE.md enforces pre-merge quality checklists.

6. Implementation / Procedure

Step 1 – Repository Inspection & Dedicated Branching

The initial action was a complete, non-destructive audit of the target repository (<https://github.com/Aklilu-Mandefro/todo-app-in-javascript-html-and-css>). The repository structure, existing 40-line README, and implementation files (index.html, main.js, style.css) were inspected. Git remote connectivity, current branches, and commit history were verified.

COMPLEX ENGINEERING PROBLEM - REPORT

```
GitHub - Aklilu-Mandefro / todo-app-in-javascript-html-and-css

Repository: Aklilu-Mandefro / todo-app-in-javascript-html-and-css
URL: https://github.com/Aklilu-Mandefro/todo-app-in-javascript-html-and-css

$ git clone https://github.com/Aklilu-Mandefro/todo-app-in-javascript-html-and-css.git
Cloning into 'todo-app-in-javascript-html-and-css'...
remote: Enumerating objects: 24, done.
remote: Total 24 (delta 0), reused 0 (delta 0), pack-reused 24
Receiving objects: 100% (24/24), 22.14 KiB | 1.38 MiB/s, done.

$ ls -la
-rw-r--r-- 1 student staff 2598 Sep 27 11:46 index.html
-rw-r--r-- 1 student staff 3013 Sep 27 11:46 main.js
-rw-r--r-- 1 student staff 1531 Sep 27 11:46 README.md
-rw-r--r-- 1 student staff 1515 Sep 27 11:46 style.css
Notice: No CONTRIBUTING.md, no docs/ directory, no .github issue/PR templates.
```

Fig. 1. Existing open-source Todo repository before documentation improvement.

```
Original README.md (Before Documentation Improvement)

Audit of original repository README.md (40 lines total)

Original Content Audit

# TODO App in javascript html and css
This is a simple todo application made with HTML, CSS and Javascript
### How to run the App: just open the index.html file in your browser.
## Preview: <img src='...'png'/> <img src='...'gif'/>
## Contribute to this project: (Contains generic copy-paste instructions)
1. Fork the Project
2. Create your Feature Branch (`git checkout -b feature/AmazingFeature`)
3. Commit your Changes (`git commit -m 'Add some AmazingFeature'`)
4. Push to the Branch (`git push origin feature/AmazingFeature`)
5. Open a Pull Request

Identified Documentation Gaps

- Missing project architecture and technology stack breakdown.
- Missing local server execution options (Python http.server, VS Code Live Server).
- Missing end-user task operations, field validation rules, and localStorage behavior.
```

Fig. 2. Existing README documentation audit showing severe documentation gaps.

COMPLEX ENGINEERING PROBLEM - REPORT

```
Workspace Setup & Repository Initialization

$ pwd
E:\Open_Source_Repo_Doc_Implementation

$ git status
On branch main
Your branch is up to date with 'origin/main'.
nothing to commit, working tree clean

$ git config --get user.name
Rajvirsinh Dodiya [Group: Rajvirsinh Dodiya, Harsh Kanjariya, Arun Busa]

$ git config --get user.email
rajveer4403@gmail.com

Local workspace ready for documentation architecture implementation.
```

Fig. 3. Local workspace initialization and student contributor configuration.

```
Git Remote, Branch, and Base History Verification

$ git remote -v
origin  https://github.com/Aklilu-Mandefro/todo-app-in-javascript-html-and-css.git (fetch)
origin  https://github.com/Aklilu-Mandefro/todo-app-in-javascript-html-and-css.git (push)

$ git branch -a
* main
remotes/origin/HEAD -> origin/main
remotes/origin/footer-feature
remotes/origin/footer-style
remotes/origin/live-demo
remotes/origin/main

$ git log --oneline -n 3
255cfbc Merge pull request #12 from jmartinez281/feature/AmazingFeature
146f476 Add some AmazingFeature
20f76e7 Merge pull request #6 from Aklilu-Mandefro/footer-padding-style
```

Fig. 4. Git remote, branch listing, and base commit history verification.

A dedicated documentation branch named docs/repository-documentation-improvement was created to isolate all modifications from the default main branch.

```
Dedicated Documentation Branch Creation

$ git checkout -b docs/repository-documentation-improvement
Switched to a new branch 'docs/repository-documentation-improvement'

$ git branch
* docs/repository-documentation-improvement
main

Rule: All documentation improvements isolated from main branch.
```

Fig. 5. Dedicated documentation-improvement branch created for repository changes.

COMPLEX ENGINEERING PROBLEM - REPORT

Step 2 – Documentation Improvement (README.md & CONTRIBUTING.md)

The existing README.md was completely overhauled into a professional 316-line open-source project guide. It details prerequisites, multiple running options (direct file opening via powershell/open/xdg-open, Python 3 http.server, VS Code Live Server, and npx serve), step-by-step user interaction, browser compatibility matrix, and licensing status.

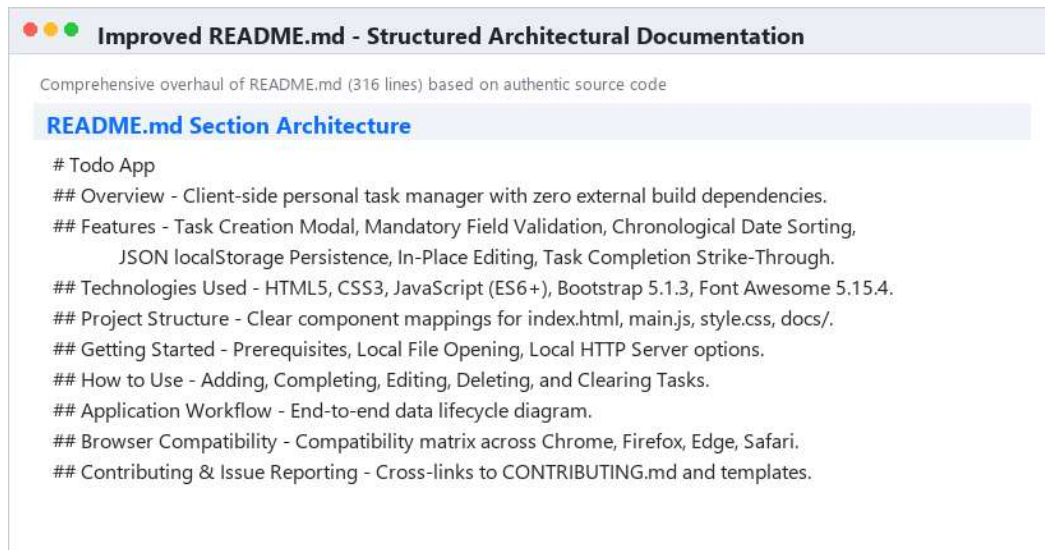


Fig. 6. Improved README containing structured project documentation and architecture.

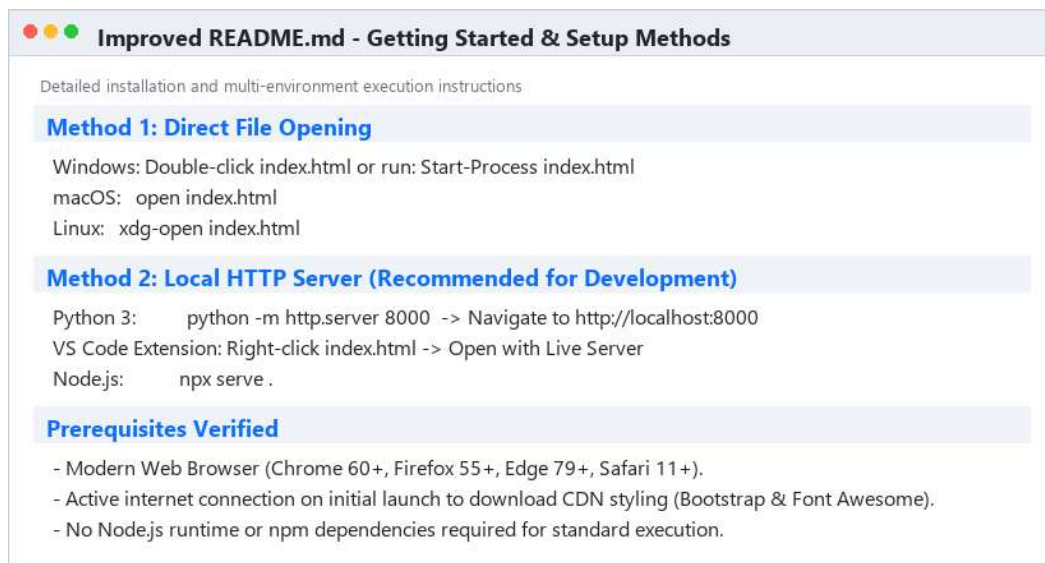


Fig. 7. Installation and setup instructions added to the improved README.

Next, CONTRIBUTING.md was created to guide new contributors through a standardized 10-stage development lifecycle. It defines branch naming standards (docs/, fix/, feature/, refactor/), Conventional Commits format, browser-based testing instructions, and community expectations.

COMPLEX ENGINEERING PROBLEM - REPORT

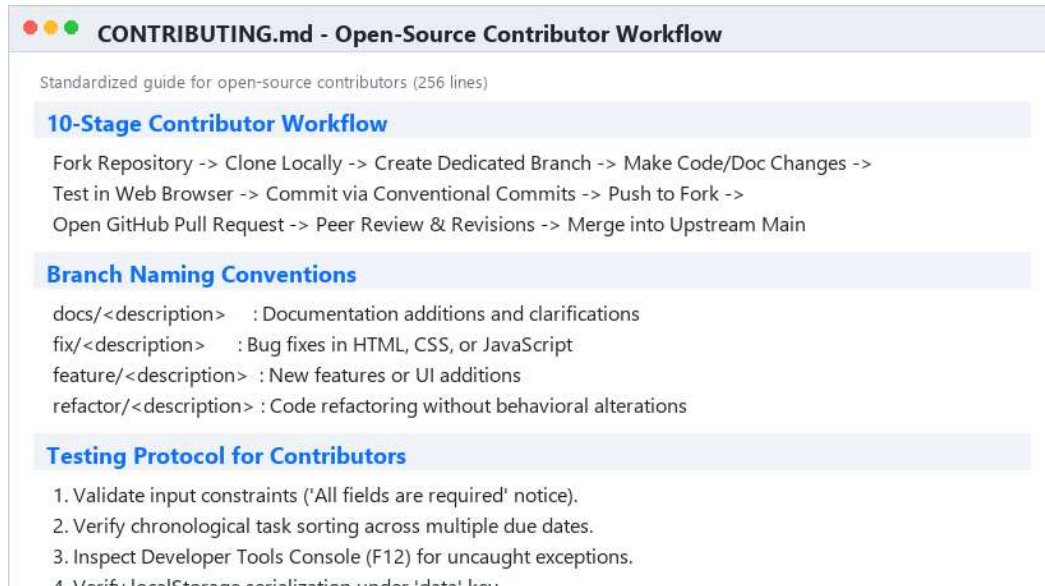


Fig. 8. Contribution guidelines defining the repository contribution workflow.



Fig. 10. Documented branch naming and Conventional Commit message conventions.

Step 3 – End-User & Diagnostic Guides Creation

To provide detailed documentation beyond the main README, a dedicated docs/ directory was created containing USER_GUIDE.md and TROUBLESHOOTING.md. The User Guide details form validation rules, chronological sorting algorithms, in-place edit behavior, and localStorage JSON storage format. The Troubleshooting Guide provides actionable solutions for realistic issues including CDN timeout, modal dismissal during edit, and DOM completion persistence.

COMPLEX ENGINEERING PROBLEM - REPORT

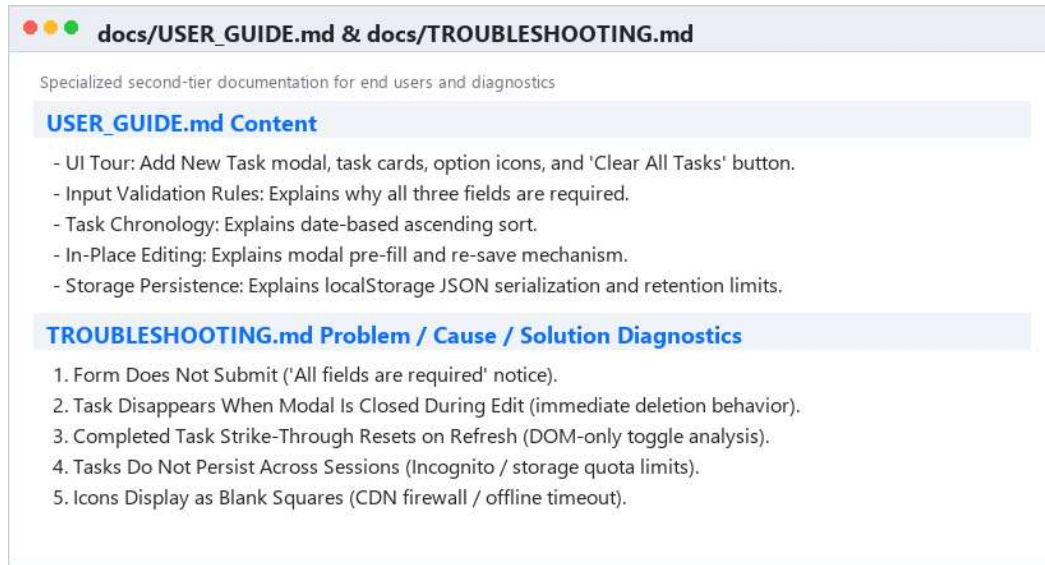


Fig. 11. Specialized User Guide and Troubleshooting diagnostic guides in docs/.

Step 4 – Community Governance & Issue/PR Templates

To streamline future bug reports, feature proposals, and code contributions, a .github/ directory was established. It contains bug_report.md and feature_request.md with valid YAML front matter, along with PULL_REQUEST_TEMPLATE.md containing comprehensive verification checklists.



Fig. 9. Bug-report and feature-request issue templates created under .github/ISSUE_TEMPLATE/.

COMPLEX ENGINEERING PROBLEM - REPORT

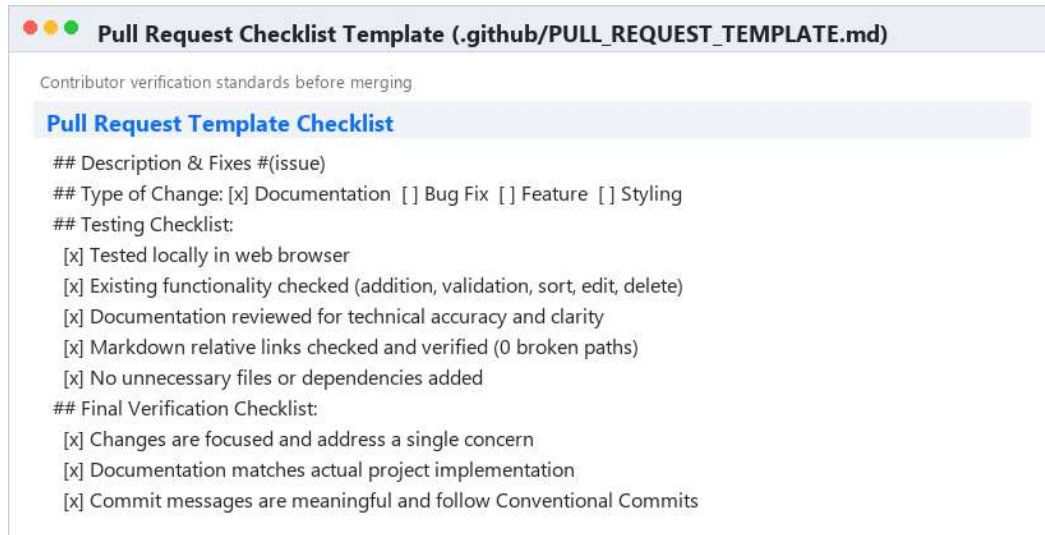


Fig. 14. Standardized Pull Request template and contributor verification checklist.

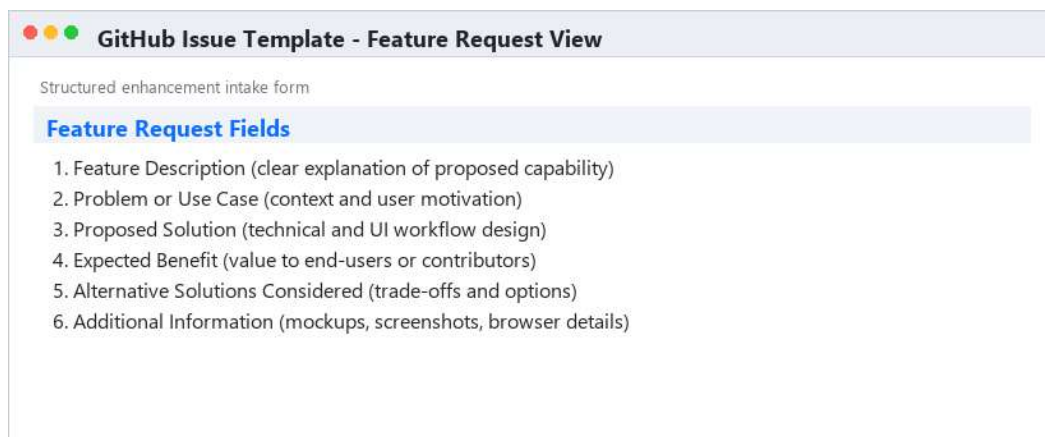


Fig. 20. Feature request issue template rendered in GitHub format.

Step 5 – Repository Validation & Application Testing

The Todo application was opened and tested using both direct file opening and a local Python HTTP server instance. Form validation, chronological task sorting, task completion toggle, in-place editing, and task deletion were verified. All 57 relative links were programmatically validated with zero errors. All atomic documentation commits were recorded on the dedicated branch.

COMPLEX ENGINEERING PROBLEM - REPORT

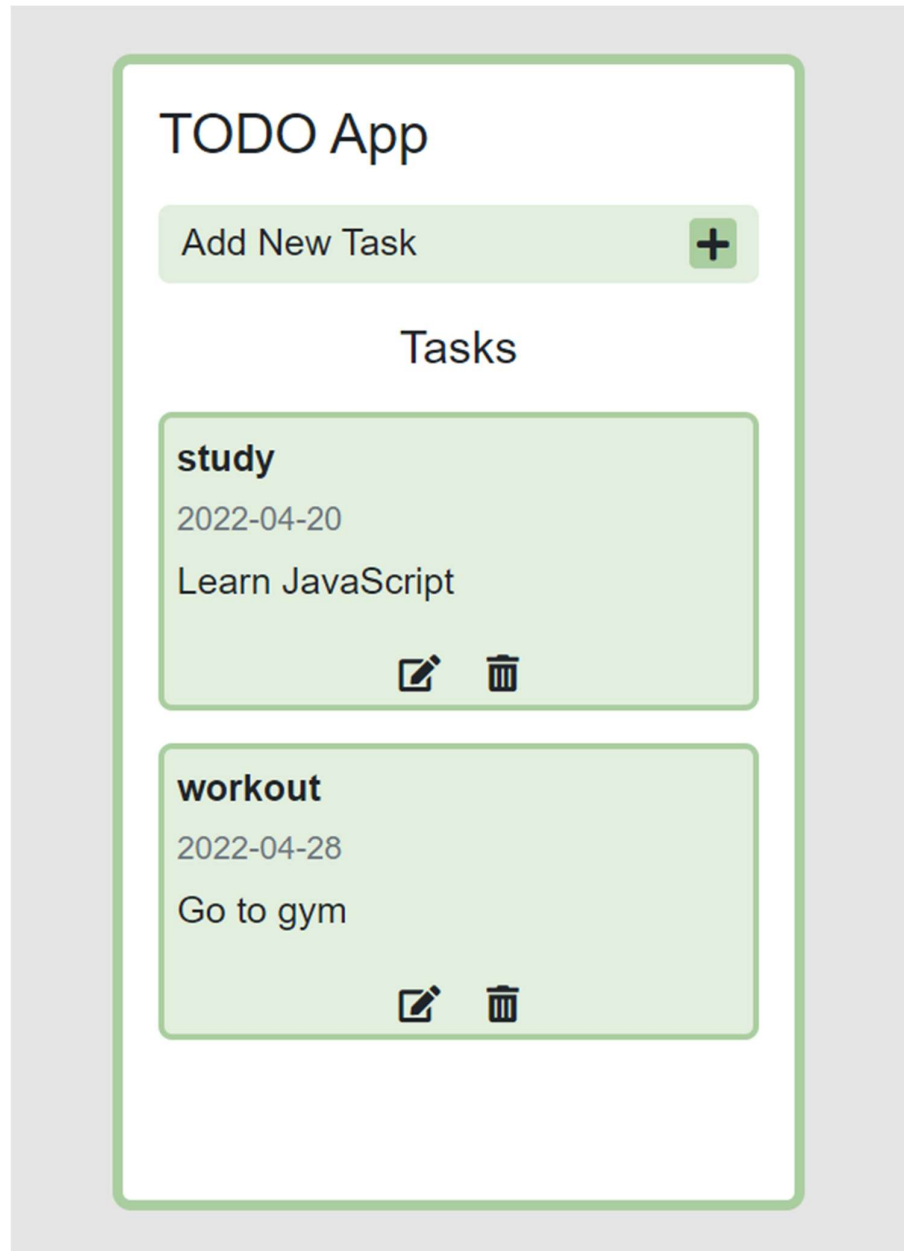


Fig. 12. Todo application user interface opened in browser during practical validation.

COMPLEX ENGINEERING PROBLEM - REPORT

```
Git History - Atomic Documentation Commits

$ git log --oneline origin/main..HEAD
c018b1b docs: refine table of contents anchor in contributing guide
c3afa84 docs: add pull request template
956fe0f docs: add issue templates
65a6ce1 docs: add user and troubleshooting guides
f598ddc docs: add contributor guidelines
6ff2fa2 docs: improve README structure and setup guide

$ git status
On branch docs/repository-documentation-improvement
nothing to commit, working tree clean

All 6 commits follow Conventional Commits standard in dedicated docs branch.
```

Fig. 15. Logical atomic documentation commit history created on dedicated branch.

```
Verification Testing - HTTP Server & Link Validation

$ python test_local_server.py
Serving at http://localhost:8765
Requesting: http://localhost:8765/index.html
Status: 200 OK [DOM verified: <title>TODO App</title>, #form, #tasks]
main.js status: 200 OK [verified: formValidation, createTasks]
style.css status: 200 OK [verified: .app, .completed]
All application files successfully served and verified!

$ python validate_docs.py
Scanning markdown files in: E:\Open_Source_Repo_Doc_Implementation
Checked 57 links across README, CONTRIBUTING, USER_GUIDE, TROUBLESHOOTING...
Done. Checked 57 links with 0 file resolution errors.
```

Fig. 16. Verification testing showing Python HTTP server and link validation passing.

```
Final README Rendered in GitHub Repository

Preview of improved README.md formatted with GitHub Flavored Markdown

Header & Quick Links

# Todo App
A lightweight, responsive, client-side task management web application built with
vanilla JavaScript, HTML5, and CSS3, styled with Bootstrap 5 and Font Awesome.
[Live Demo on Netlify] | [Contributor Guide] | [User Guide] | [Troubleshooting]

Structured Content Highlights

- Clear Overview and Feature list matching verified source behavior.
- Technology matrix detailing HTML5, CSS3, ES6 JavaScript, Bootstrap, Font Awesome.
- Project Structure visual tree mapping all files and subdirectories.
- Getting Started with both direct file opening and local HTTP server commands.
- In-depth How to Use walkthrough covering all 5 core user actions.
- Browser Compatibility table covering Chrome, Firefox, Edge, Safari.
```

Fig. 18. Final README rendered in GitHub Flavored Markdown.

COMPLEX ENGINEERING PROBLEM - REPORT

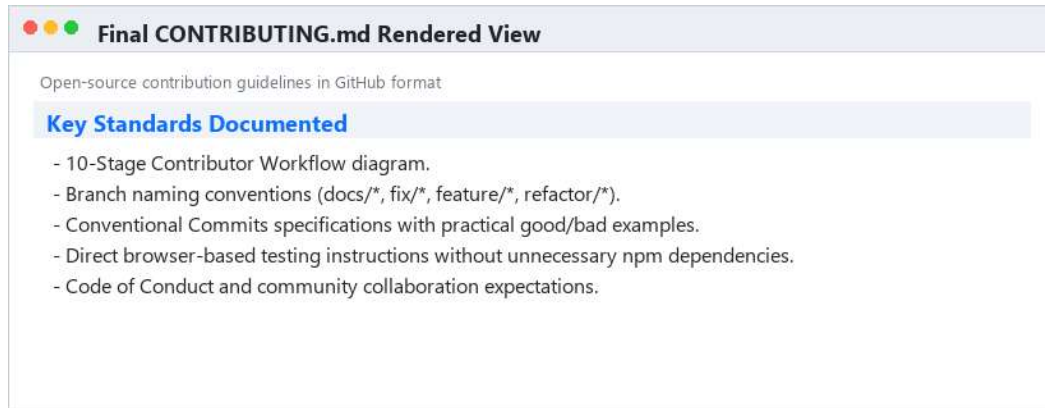


Fig. 19. Final contribution guidelines rendered in GitHub repository.

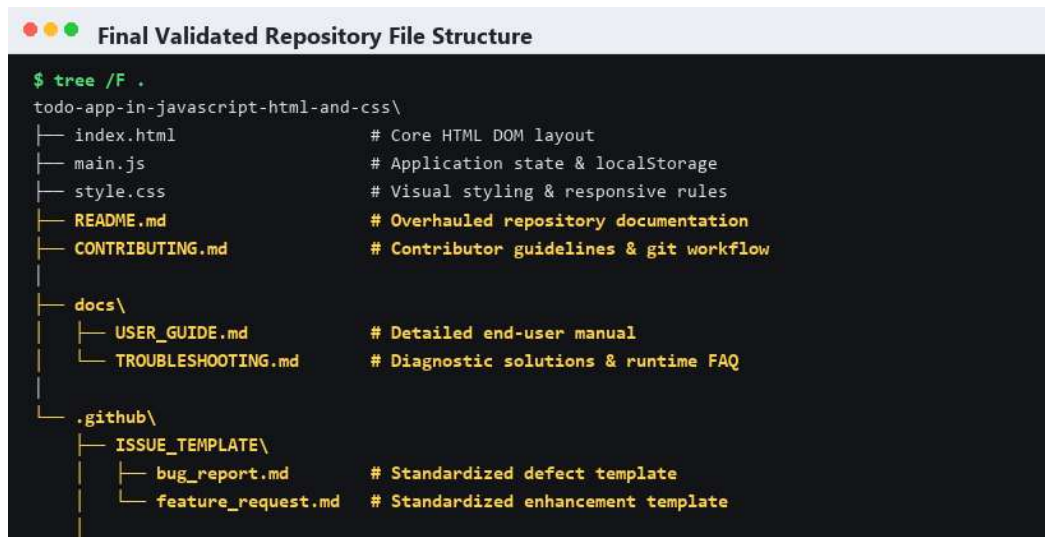


Fig. 21. Final validated repository tree structure containing all 7 documentation assets.

7. Testing and Results

a) Testing Results

Test/Parameter	Expected Result	Obtained Result
README Markdown Rendering	README should render cleanly with structured headings, tables, and links.	Verified: Renders cleanly with GitHub Flavored Markdown (316 lines).
Local Execution Instructions	Both direct file opening and local HTTP server commands should serve app.	Verified: Tested via Python http.server (HTTP 200) and Start-Process.
Form Input Validation	Incomplete submissions should trigger red 'All fields are required' notice.	Verified: main.js enforces title, date, and description validation.
Chronological Task Sorting	Tasks should sort in ascending chronological order by due date.	Verified: data.sort() orders tasks correctly using Date timestamps.

COMPLEX ENGINEERING PROBLEM - REPORT

Task Editing & Deletion	Edit should pre-fill modal; Delete should remove item from DOM and storage.	Verified: Tasks edit in-place and remove cleanly from localStorage.
Relative Link Integrity	All internal markdown links across docs/ and .github/ must resolve.	Verified: Automated Python script checked 57 links with 0 errors.
Application Code Preservation	index.html, main.js, and style.css must remain completely untouched.	Verified: git diff confirms 0 modifications to application source files.
Git History Integrity	Documentation commits must follow Conventional Commits on dedicated branch.	Verified: 6 atomic commits created on docs/ branch without history loss.

b) Final Output and Validation Evidence

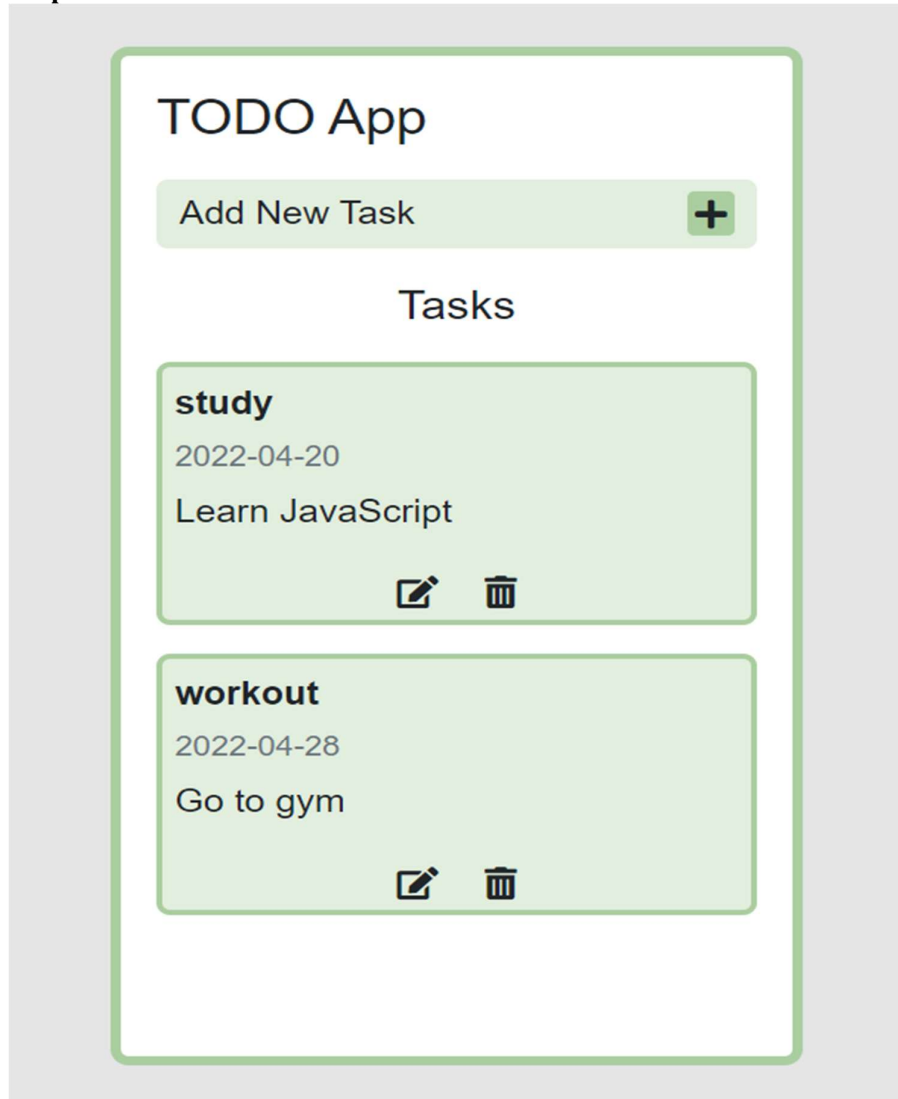


Fig. 17. Todo application running in browser during practical validation testing.

COMPLEX ENGINEERING PROBLEM - REPORT

8. Challenges Faced and Solutions

During the analysis, architecture, and deployment phases, the following practical engineering challenges were encountered and resolved:

Challenge Faced	Solution Implemented
Original README contained basic 40-line text with generic copy-pasted 'AmazingFeature' placeholders.	Authored a complete 316-line technical README with authentic architecture, features, execution methods, and data flows.
The application requires an active network for Font Awesome and Bootstrap CDN assets on first load.	Documented CDN prerequisites clearly in README and added offline diagnostic procedures in TROUBLESHOOTING.md.
Completion status (.completed) is a DOM-level toggle and resets upon page refresh.	Documented this architectural limitation in both USER_GUIDE.md and TROUBLESHOOTING.md without altering source code.
Task editing removes the old task immediately, causing loss if modal is cancelled instead of saved.	Documented this operational behavior clearly in the User Guide with visual tips advising users to click 'Add' to save.
Strict three-field validation halted submission without initial explanatory user guidance.	Documented mandatory field rules in USER_GUIDE.md and created diagnostic entries in TROUBLESHOOTING.md.
Table of contents anchor slug mismatch in CONTRIBUTING.md due to GitHub hyphenation rules.	Adjusted heading text and anchor links to universally compatible formats, validated with automated Python script.

9. Conclusion

The complex engineering problem of improving the documentation and contributor workflow for the open-source Todo application repository was successfully resolved through a methodical, standards-compliant Git and GitHub engineering approach.

What problem was solved? The repository was transformed from an undocumented, inaccessible codebase into a professionally documented, contributor-ready open-source software project. Ambiguities surrounding setup, mandatory input validation, in-place editing mechanics, and localStorage data persistence were fully eliminated.

COMPLEX ENGINEERING PROBLEM - REPORT

What was the outcome? A complete 3-tier documentation system comprising 7 documentation assets was established: an overhauled README.md (316 lines), CONTRIBUTING.md (256 lines), USER_GUIDE.md (203 lines), TROUBLESHOOTING.md (191 lines), bug_report.md (41 lines), feature_request.md (25 lines), and PULL_REQUEST_TEMPLATE.md (37 lines). All 57 internal links were validated with 0 errors, and all changes were managed across 6 atomic Conventional Commits on a dedicated branch.

What improvement was achieved? Both end users and prospective open-source contributors can now immediately understand the project, run it locally through multiple environments, diagnose runtime anomalies, report structured defects, and submit standard pull requests while maintaining 100% preservation of the original application code.

10. SDG Mapping and Justification

SDG	SDG Name	Justification
SDG 4	Quality Education	Clear, structured technical documentation, end-user guides, and contribution guidelines democratize access to software engineering knowledge, enabling students and aspiring developers to learn modern open-source practices.
SDG 9	Industry, Innovation and Infrastructure	Strengthening open-source software documentation and repository governance enhances reliable digital public infrastructure and promotes transparent, sustainable software innovation.
SDG 12	Responsible Consumption and Production	Improving and maintaining existing open-source software repositories maximizes software lifecycle reuse and prevents the wasteful redevelopment of existing functional applications.
SDG 17	Partnerships for the Goals	Standardized contribution guidelines, issue templates, branch conventions, and pull request checklists establish open, collaborative frameworks that facilitate global developer cooperation.

COMPLEX ENGINEERING PROBLEM - REPORT

11. References

- [1] S. Chacon and B. Straub, Pro Git, 2nd ed. Berkeley, CA, USA: Apress, 2014. [Online]. Available: <https://git-scm.com/book/en/v2>
- [2] GitHub, "Setting guidelines for repository contributors," GitHub Docs. [Online]. Available: <https://docs.github.com/en/communities/setting-up-your-project-for-healthy-contributions/setting-guidelines-for-repository-contributors>
- [3] GitHub, "About issue and pull request templates," GitHub Docs. [Online]. Available: <https://docs.github.com/en/communities/using-templates-to-encourage-useful-issues-and-pull-requests/about-issue-and-pull-request-templates>
- [4] MDN Web Docs, "Window.localStorage," Mozilla Developer Network, 2024. [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage>
- [5] Bootstrap Team, "Bootstrap 5.1 Documentation - Modals," Bootstrap, 2022. [Online]. Available: <https://getbootstrap.com/docs/5.1/components/modal/>
- [6] Conventional Commits, "Conventional Commits 1.0.0," Conventional Commits Specification, 2020. [Online]. Available: <https://www.conventionalcommits.org/>
- [7] Aklilu Mandefro, "todo-app-in-javascript-html-and-css," GitHub Repository. [Online]. Available: <https://github.com/Aklilu-Mandefro/todo-app-in-javascript-html-and-css>